

✓ Lab 5: Digital Signal Processing Number Systems

Jonathan Sumner, Diego Lopez, Rachel DuBois, and Argenis Aquino, *Department of Engineering Technology, Rochester Institute of Technology*

□

Abstract— This paper explores the effects of different data types (byte, signed integer, long integer, and float) on memory usage, computation speed, and numerical accuracy in digital signal processing (DSP) applications. The experiment investigates how finite precision arithmetic impacts calculations, particularly in scenarios involving addition, subtraction, and convolution operations. By measuring execution times and analyzing roundoff errors, the study demonstrates the trade-offs between precision and performance when using various data types. Additionally, the experiment examines the accumulation of errors in floating-point arithmetic and evaluates the effectiveness of moving average filters in DSP. The results highlight the importance of selecting appropriate data types to balance computational efficiency and numerical accuracy in real-time signal processing systems.

I. Introduction

Digital signal processing (DSP) relies heavily on numerical computations, where the choice of data types can significantly impact both performance and accuracy. In embedded systems, such as those using Arduino microcontrollers, memory and processing power are limited, making it crucial to optimize data representation and arithmetic operations. This lab investigates the behavior of different data types—byte, signed integer, long integer, and floating-point—in DSP applications, focusing on their effects on memory usage, computation speed, and numerical precision. One of the key challenges in DSP is managing finite precision arithmetic, which can lead to round-off errors and unexpected behavior in calculations. For example, floatingpoint numbers, while offering high precision, can introduce errors due to their limited mantissa and exponent representation. Similarly, integer data types, such as byte and long, can experience overflow or underflow when performing arithmetic operations, leading to incorrect results. Understanding these limitations is essential for designing efficient and accurate DSP systems.

II. METHODOLOGY and Results

A. Basic Math Operations and Data Types (Section One)

In this section, we repeatedly added two numbers of varying sizes together, making use of various common data types to witness how results were impacted.

```
Lab5_Section1_done
void setup()
{
  Serial.begin(115200);
  Serial.println(F("Lab 5 datatypes"));
  Serial.println(F("\nEnter 'g' to go ....."));
  while (Serial.read() != 'g'); // spin until 'g' entry

  Serial.println("Data type is int"); // byte, int, long, float
  av = 100;
  bv = 100;
  yv = av + bv;
  Serial.println("Add two numbers (100+100)");
  Serial.print("a = "); Serial.print(av); Serial.print("\t"); Serial.print("\t");
  Serial.print("b = "); Serial.print(bv); Serial.print("\t"); Serial.print("\t");
  Serial.print("y = a+b = "); Serial.println(yv);

  av = 200;
  bv = 200;
  yv = av + bv;
  Serial.println("\nAdd two numbers (200+200)");
  Serial.print("a = "); Serial.print(av); Serial.print("\t"); Serial.print("\t");
  Serial.print("b = "); Serial.print(bv); Serial.print("\t"); Serial.print("\t");
  Serial.print("y = a+b = "); Serial.println(yv);

  av = 20000;
  bv = 20000;
  yv = av + bv;
  Serial.println("\nAdd two numbers (20,000+20,000)");
  Serial.print("a = "); Serial.print(av); Serial.print("\t"); Serial.print("\t");
  Serial.print("b = "); Serial.print(bv); Serial.print("\t"); Serial.print("\t");
  Serial.print("y = a+b = "); Serial.println(yv);
}

void loop() {
  Serial.println("\nAdd numbers from array");

  // TODO: Add a for-loop to run through all the 'a' and 'b' values in the
  // arrays 'av1' and 'bv1'. Start at index 0 and run through all 16 values in the array.
  // Inside the for-loop:
  // 1) set av and bv to values picked from 'av1' and 'bv1'
  // 2) perform the addition
  // 3) print the a and b values and the result yv

  // INSERT YOUR CODE HERE
  for(int i = 0; i < 16; i++){
    av = av1[i];
    bv = bv1[i];
    yv = av + bv;
    Serial.print("a = "); Serial.print(av); Serial.print("\t"); Serial.print("\t");
    Serial.print("b = "); Serial.print(bv); Serial.print("\t"); Serial.print("\t");
    Serial.print("y = a+b = "); Serial.println(yv);
  }
}
```

Figure 1: Section 1 completed code.

After editing the code as required, we would run it in Serial Monitor on an Arduino to retrieve the values to put into Table 1a. This code would be re-run for each data type; for byte, int, long, and float, in order to fill all of the table's requested values.

Input a	Input b	Expected result a+b	Actual Result byte	Actual Result int	Actual Result long	Actual Result float
100	100	200	200	200	200	200
200	200	400	144	400	400	400
20,000	20,000	40,000	64	-25536	40000	40000
100	155	255	255	255	255	255
100	156	256	0	256	256	256
100	157	257	1	257	257	257
255	100	355	99	355	355	355
255	10	265	9	265	265	265
25,500	25,500	51,000	56	-14536	51000	51000
20,000	12,767	32,767	255	32767	32767	32767
20,000	12,768	32,768	0	-32768	32768	32768
20,000	12,769	32,769	1	-32767	32769	32769
20,000	12,770	32,770	2	-32766	32770	32770
1,000,000,000	1,000,000,000	2,000,000,000	0	-27648	2000000000	2000000000
2,000,000,000	147,483,649	2147483649	1	1	-2147483647	2147483648
2,000,000,000	147,483,649	2147483649	1	1	-2147483647	2147483648

Table 1a: Various data types in basic addition calculations.

For the next major step in this section, in order to ensure we fully understood why some data types would overflow, we had to fill in Table 1b and obtain specific values with either byte or long.

Input a	Find Positive number b	Expected result a+b	Actual Result byte	Actual Result int	Actual Result long	Actual Result float
255	255	//skip//	254	//skip//	//skip//	//skip//
2,000,000,000	2294967286	//skip//	//skip//	//skip//	-10	//skip//

Table 1b: Calculating a specific overflowed value.

We noticed that some of the expected values we calculated did not match up with the actual calculated values when we ran each section of the code. This error was able to be small or very large, or not at all. This was due to some data types having an overall smaller capacity for values, which is able to be seen in the tables. Sometimes a decimal place would also be appended, but this was overall a null issue since it would have no change on the overall value, unlike overflowing the data type.

B. Illustration of Roundoff Error with Floats (Section 2)

In this section, we compared ideal output values and actual output values in order to see how error changes as we scale up the factor for random numbers. This was outputted to Table 2. Only the float data type was used. How it works was from adding two random numbers together to a constant and subtract from the initial value. The roundoff error describes the relationship between our expected error and the actual error we get.

```
const int NUM_CALC = 1000;
float xv = 0.0, A, B, error, scale_factor;

void setup()
{
  Serial.begin(115200);
  Serial.println(F("Lab 3 P2 datatypes 190928"));
  Serial.println(F("\nEnter 'g' to go ....."));
  while (Serial.read() != 'g'); // spin until 'g' entry

  randomSeed(425);

  scale_factor = 100000.0; // start with 1.0
  xv = 1.0;
  Serial.print("Adding random floating point numbers with scale factor ");
  Serial.println(scale_factor);
  Serial.print("x value starts at x = ");
  Serial.println(xv,10); // print to 10 decimal places
  for (int i = 1; i < NUM_CALC; i++)
  {
    A = random_float() * scale_factor;
    B = random_float() * scale_factor;
    xv = xv+A; // add A to x
    xv = xv+B; // add B to x
    xv = xv-A; // subtract A from x
    xv = xv-B; // subtract B from x
  } // for

  error=(1.0-xv);

  // end result should just be the original value of x (i.e. 1.0)
  Serial.print("x value finishes at x = "); Serial.println(xv,10);
  Serial.print("Total percent error after 1000 calculations = ");
  Serial.print(error*100.0,10); Serial.println(" %");
  Serial.print("Average percent error per addition or subtraction = ");
  Serial.print(error*100.0/(4.0*NUM_CALC),10); Serial.println(" %");
  Serial.println();
} // setup()

void loop() {} // spin forever

//*****
float random_float() { return (random(2147483648)/2147483648.0); }
```

Figure 2: Section 2 code.

Using the code and various scale values, we could see that the error could become larger or smaller in the table for section 2.

Scale factor for random numbers	Starting value of x	Ending value of x	The total error in percent after 1000 calculations	Average error in percent for one calculation
1	1	0.999996948	0.0003000000%	0.000000760%
10	1	0.999987602	0.0012397766%	0.0000003099%
100	1	1.000007629	-0.0000076293%	-0.0000001907%
1000	1	1.000427246	-0.0427246093%	-0.0000106812%
10000	1	0.984375	1.5625000000%	0.0003906250%
100000	1	1.0546875	-5.4687500000%	-0.0013671875%

Table 2: Roundoff Error calculations with Floats.

We also can see that the signal to noise ratio we obtained was 1.0546875 for a scale factor of 10,000. The reason that the sizes of A and B impact the result we get is because they are multiplied by the scale factor, so when it is larger, as are A and B; which leads to the ending value increasing too.

One case where floating point numbers would not be accurate is when using binary values. An example is when using the number 0.1 which does not have a binary value. This means that when doing arithmetic where 0.1 is multiplied the result may not be what is expected.

This issue is caused by the finite precision of floating point numbers. When certain values that require precision undergo iterative arithmetic issues come up. Issues such as rounding and truncation of the value will cause inconsistencies and precision errors in the data.

C. Unexpected Behavior Due to Finite Precision Effects with Floats (Section 3)

Here, we look over how finite precision associated with floating point data types is able to cause unexpected results when used in calculations. The code used here has values A and B one number apart, and will print the difference between the two. We then uses A and B to count through a loop, which we let run for a bit before analysis.

```
float A = 18000002.0;
float B = 18000001.0;
//long A = 18000002;
//long B = 18000001;
//long A = 18000002.0;
//long B = 18000001.0;

void setup()
{
  Serial.begin(115200);
  Serial.print("A = float(18000002.0) value printed out is "); Serial.println(A);
  Serial.print("B = float(18000001.0) value printed out is "); Serial.println(B);
  Serial.print("A - B is "); Serial.println(A-B);

  //-----
  Serial.println("\nAnother illustration: Counting up by 1 ");
  Serial.println("A is the counter value");
  Serial.println("B is the counter value minus 1");
  Serial.println();

  long counter=16000000;
  A = counter;
  B = A-1.0;
  Serial.print("Starting at a counter value of = "); Serial.println(counter);
  Serial.print("A value is "); Serial.println(A);
  Serial.print("B value is "); Serial.println(B);
  Serial.print("A - B is "); Serial.println(A-B);

  Serial.println();
  Serial.println("Continue to count up by 1. Print the counter every 50,000 counts");
  Serial.println("Check to see that the difference between A and B is always 1, error otherwise");
  Serial.println();
  while ((A-B) == 1)
  {
    if (counter%50000 == 0) Serial.println(counter);
    counter = counter+1;
    A=counter;
    B=counter-1;
  }

  Serial.print("\nUnexpected result: Counter = ");
  Serial.println(counter);
  Serial.print("A value is "); Serial.println(A);
  Serial.print("B value is "); Serial.println(B);
  Serial.print("A - B is "); Serial.println(A-B);
} // setup

void loop() {} // spin forever
```

Figure 3: Section 3 code.

The first thing we noticed was that B never printed out correctly, which was due to it being a single precision floating point number, which would end up with it being rounded down to 18000000.0, as opposed to its actual set value.

In the second section of the code, we hit an error at 16777217, which happens because we exceeded the mantissa value which gave us a range of 23 bits total. We tried setting it to the data type long and got different results, where A – B would compute properly as 1, and the program would not stop counting since it would never end up with a time where A – B was equal to 0.

However, we did have to remove the decimal points from A and B since the long data type is a fixed point number, so there is no ability to represent a decimal value.

D. Finite Precision Effects on Sinewave Calculations (Section 4)

For this section, we looked over how floating point number systems can be seen when calculating sine and cosine functions. We had to run the code and analyze the results that it spit out in Excel.

```
#include "Math.h" // for accessing sine function

float result;
float phi = 0.0;

void setup()
{
  Serial.begin(115200);

  Serial.println(F("Lab 5 P4 datatypes\n"));
  Serial.println(F("Enter 'g' to go ....."));
  while (Serial.read() != 'g'); // spin until 'g' entry

  Serial.println("Prints values of sin(2*pi*i) where i is integer");
  Serial.println("Theoretically, the value should always be zero");
  Serial.print("\ni\tsine(n2pi)\n");

  // Iterate from 0 to 999 by one. Compute an angle value
  // that is 2*pi*i. Find the SINE of that value.

  for (int i = 0; i < 1000; i+=8)
  {
    phi = TWO_PI * i; // TWO_PI is Arduino constant in Arduino.h
    result=sin(phi);

    // Print the results to the serial port

    Serial.print(i); Serial.print('\t');
    Serial.print(phi,20); Serial.print('\t');
    Serial.println(result,20); // print 20 decimal places
  }
} // setup

void loop() {} // spin forever
```

Figure 4: Section 4 code.

After running the code, we went to excel to make a plot of the results, which looked like the following graphs.

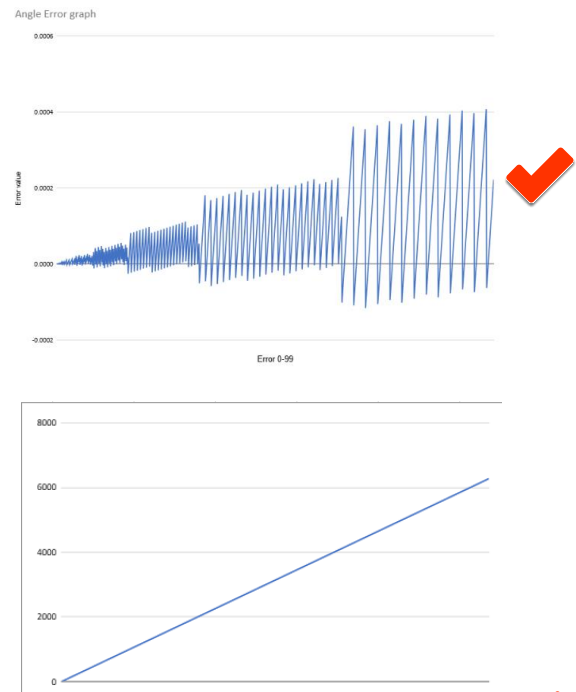


Figure 5: Angle vs Error graphs.

This graph enables us to visualize the learning outcomes of this section much easier. These graphs were created using the output which can be seen in Table 3.

0	0	0
1	6.283185482	0.000000175088
2	12.56637096	0.000000350177
3	18.84955597	0.000000048428
4	25.13274193	0.000000700354
5	31.41592789	0.000001352280
6	37.69911194	0.000000096857
7	43.9822998	0.000002656131
8	50.26548386	0.000001400709
9	56.54866791	0.000000145286
10	62.83185577	0.000002704560
11	69.11504364	0.000005263834
12	75.39822388	0.000000193715
13	81.68141174	0.000002752989
14	87.96459961	0.000005312263
15	94.24777985	0.000000242143
16	100.5309677	0.000002801418
17	106.8141556	0.000005360692
18	113.0973358	0.000000290572
19	119.3805237	0.000002849847
20	125.6637115	0.000005409121
21	131.9468994	0.000007968396
22	138.2300873	0.000010527670
23	144.5132599	-0.00000217184
24	150.7964478	0.000000387430
25	157.0796356	0.000002946704
26	163.3628235	0.000005505979
27	169.6460114	0.000008065253
28	175.9291992	0.000010624527
29	182.2123718	-0.00000207498
30	188.4955597	0.000000484287
31	194.7787476	0.000003043562
32	201.0619354	0.000005602836
33	207.3451233	0.000008162111
34	213.6283112	0.000010721385
35	219.911499	0.000013280659
36	226.1946716	0.000000581145
37	232.4778595	0.000003140419
38	238.7610474	0.000005699694
39	245.0442352	0.000008258968
40	251.3274231	0.000010818243
41	257.6105957	-0.00000188127
42	263.8937988	0.000015936791
43	270.1769714	0.000003237277
44	276.4601746	0.000021055340
45	282.7433472	0.000008355826
46	289.0265198	-0.00000434368
47	295.3097229	0.000013474375
48	301.5928955	0.000000774860
49	307.8760986	0.000018592924
50	314.1592712	0.000005893405

angle argument increases in value greatly as it does here, there would be a concern regarding the error eventually, but since the error is generally so small, even at 2000 times pi, for what we would be using it for in class we are not all that concerned about error. The value of the sine generated by the Arduino at 2pi is considered accurate since the error is smaller, but the error both ways is extremely small. For the best accuracy, a user should stay between zero and 2pi, since the values should repeat for most standard sine functions.

Table 3: Section of Error vs Angle Table.

We can see in this table that the size of the error when it is at 2pi, which is sample 1, is 0.00000017508864402770. We also can see at 2000pi that the error becomes 0.00022212042808532714. At 0, every other value is also 0. The errors are not the same size because as the values were multiplied more, the error compounded into the results, creating a larger error value overall. Ultimately, if we were generating a digital sine wave, and the

E. Measuring Execution Time and Memory Usage Vs. Data Type (Section 6)

In this section, we measured the time it took for each section of code to run. We would do different operations and see how much time it took to run based on the data type in question. We did this for byte, int, long, and float. Alongside this, we had to test the different speeds it took in different implementations. The implementations used were individual operations, like addition and multiplication, alongside utilizing each operation inside of a for loop, and arrays. This all gave us what we gathered for Table 3.

```

1 const int DATA_LEN = 200; // number of data array values
2 unsigned long startTime, endTime, totalTime;
3 byte a[DATA_LEN] = {0}, sum = 1.0; // array
4 // initialized to zero
5
6 void setup()
7 {
8   Serial.begin(115200);
9   Serial.print("Data array length = ");
10  Serial.println(DATA_LEN);
11
12  res = sum;
13  startTime = micros();
14  res = res + sum; res = res + sum; res = res + sum;
15  res = res + sum; res = res + sum; res = res + sum;
16  res = res + sum; res = res + sum; res = res + sum;
17  res = res + sum; res = res + sum; res = res + sum;
18  endTime = micros();
19  Serial.print("Time per individual addition = ");
20  Serial.println((endTime - startTime) / 1000.0);
21
22  // .....
23  res = sum;
24  startTime = micros();
25  res = res * res; res = res * res; res = res * res;
26  res = res * res; res = res * res; res = res * res;
27  res = res * res; res = res * res; res = res * res;
28  res = res * res; res = res * res; res = res * res;
29  endTime = micros();
30  Serial.print("Time per individual multiplication = ");
31  Serial.println((endTime - startTime) / 1000.0);
32
33  // .....
34  res = sum;
35  startTime = micros();
36  res = res + res; res = res + res; res = res + res;
37  res = res + res; res = res + res; res = res + res;
38  res = res + res; res = res + res; res = res + res;
39  res = res + res; res = res + res; res = res + res;
40  endTime = micros();
41  Serial.print("Time per individual multiply and
42  accumulate = ");
43  Serial.println((endTime - startTime) / 1000.0);
44
45  // .....
46  res = sum;
47  startTime = micros();
48  for (int i=0; i < DATA_LEN; i++)
49  {
50    res = res + res;
51    Serial.print(i); Serial.print(" ");
52    Serial.println(res);
53  }
54  endTime = micros();
55  Serial.print("Time per loop addition = ");
56  Serial.println((float)(endTime - startTime) / DATA_LEN);
57
58  // .....
59  // multiply two values
60  startTime = micros();
61  for (int i=0; i < DATA_LEN; i++) { res = res * res; }
62  endTime = micros();
63  Serial.print("Time per loop multiplication = ");
64  Serial.println((float)(endTime - startTime) / DATA_LEN);
65
66  // .....
67  // multiply two values and add to original value
68  // (accumulate)
69  startTime = micros();
70  for (int i=0; i < DATA_LEN; i++) { res = res * res; }
71  endTime = micros();
72  Serial.print("Time per loop multiply and accumulate = ");
73  Serial.println((float)(endTime - startTime) / DATA_LEN);
74
75  // .....
76  // add two values using arrays
77  startTime = micros();
78  for (int i=0; i < DATA_LEN; i++) { a[i] = a[i] + a[i]; }
79
80  endTime = micros();
81  Serial.print("Time per loop array addition = ");
82  Serial.println((float)(endTime - startTime) / DATA_LEN);
83
84  // .....
85  // multiply two values using arrays
86  startTime = micros();
87  for (int i=0; i < DATA_LEN; i++) { a[i] = a[i] * a[i]; }
88
89  endTime = micros();
90  Serial.print("Time per loop array multiplication = ");
91  Serial.println((float)(endTime - startTime) / DATA_LEN);
92
93  // .....
94  // multiply two values and add to original value
95  // (accumulate) using arrays
96  startTime = micros();
97  for (int i=0; i < DATA_LEN; i++) { a[i] = a[i] + a[i] * a[i]; }
98
99  endTime = micros();
100 Serial.print("Time per loop array multiply and
101 accumulate = ");
102 Serial.println((float)(endTime - startTime) / DATA_LEN);
103
104 // void loop() { // spin forever

```

Figure 6: Section 5 Code.

Summary Table				
Datatype	byte	int	long	float
Free Memory (bytes)	1285	1084	682	682
bytes per variable for given datatype	763	964	1366	1366
SPEED USING INDIVIDUAL OPERATIONS				
microseconds per addition	0.96	0.2	0.2	0.2
microseconds per multiply	1.04	0	0.2	3
microseconds per multiply and accumulate	1.12	0.2	0.2	10.2
Speed multiplier for addition	0.192	0.04	0.04	0.272
Speed multiplier for multiply and accumulate	11.424	2.04	2.04	15.504
SPEED USING FOR LOOP (200 iterations, DATA_LEN = 200)				
microseconds per addition	0.34	0.44	0.56	6.96
microseconds per multiply	0.56	0.92	5.5	7.76
microseconds per multiply and accumulate	0.56	1.26	5.88	16.18
Speed multiplier for addition	2.3664	3.0624	3.8976	9.3264
Speed multiplier for multiply and accumulate	9.0608	20.3868	95.1384	124.586
SPEED USING FOR LOOP and ARRAY Storage (200 iterations, DATA_LEN = 200)				
microseconds per addition	0.96	1.32	1.98	7.82
microseconds per multiply	1.04	1.58	6.8	6.98
microseconds per multiply and accumulate	1.12	2.02	7.14	13.24
Speed multiplier for addition	7.5072	10.3224	15.4836	33.3132
Speed multiplier for multiply and accumulate	14.8288	26.7448	94.5336	136.1072

Table 3: Operation time depending on the data type used.

From this table, we can see that some data types run a bit faster or slower depending on how they are used, and what operation they are used for.

F. Convolution Implementation with Measuring Execution Time (Section 6)

Finally, for this section, we modified code to learn more about convolution. We modified code to give us the results that are stored in the table below, where we plot the update rate and execution time to answer the following questions.

```

unsigned long startTime, endTime, execTime; // micros() casting
const int IMP_RESP_LEN = 30;
const int DATA_LEN = 100;

// initialize all arrays to zero
float xv[DATA_LEN] = {0}; // convolution input: x variable
float yv[DATA_LEN] = {0}; // convolution output: y variable
float h[IMP_RESP_LEN] = {0}; // filter impulse response

void setup()
{
  Serial.begin(115200);
  Serial.println("x variable\n\n\txv[n]");
  for (int i = 0; i < DATA_LEN; i++)
  {
    xv[i] = 5.0;
    Serial.print(i); Serial.print('\t');
    Serial.println(xv[i]);
  }

  Serial.println("\nh impulse response\n\n\th[n]");
  for (int i = 0; i < IMP_RESP_LEN; i++)
  {
    h[i] = 1.0/IMP_RESP_LEN;
    Serial.print(i); Serial.print('\t');
    Serial.println(h[i], 4);
  }

  // perform sum of products
  // start convolution only where data is valid
  // first IMP_RESP_LEN-1 datapoints are not valid

  startTime = micros();
  for (int k = IMP_RESP_LEN-1; k < DATA_LEN; k++){
    for (int i = 0; i < IMP_RESP_LEN; i++){
      yv[k] = yv[k]+h[i]*xv[k-i];
    }
  }
  endTime = micros()-startTime;

  Serial.println("\n\n\txv\tyv");
  for (int i = 0; i < DATA_LEN; i++)
  {
    Serial.print(i); Serial.print('\t');
    Serial.print(xv[i], 4); Serial.print('\t');
    Serial.println(yv[i], 4);
  }

  Serial.println("\n\nFor given data type:");
  Serial.println("-----");
  Serial.print("Data length = ");
  Serial.println(DATA_LEN);
  Serial.print("final value of yv = ");
  Serial.println(yv[DATA_LEN-1]);
  Serial.print("microseconds for each new datapoint = ");
  Serial.println(execTime/(DATA_LEN-(IMP_RESP_LEN-1)));
  Serial.print("Max update rate (Hz) = ");
  Serial.println(1000000/(execTime/(DATA_LEN-(IMP_RESP_LEN-1))));
}

void loop(){ } // spin forever

```

Figure 7: Section 6 Code.

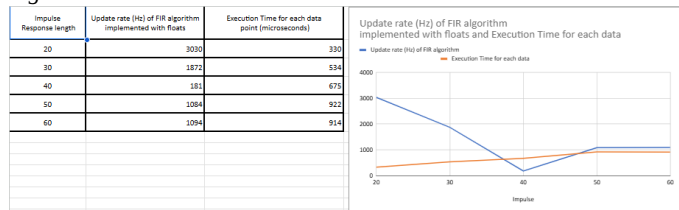


Figure 7: Convolution Update Rate table and graphs.

The data in the table shows how changing the datatype affects memory and processing speed. By changing the datatype of the variables it is shown that each data type has some places where it is able to take more or less time to run.

As the length of the impulse response increases, the execution time per data point also increases, which slows down the update rate of the FIR algorithm. This is because a longer impulse response requires more computations per sample, leading to higher processing delays. For example, when the impulse response length is 30, the update rate

is 1,872 Hz, meaning the algorithm can process 1,872 samples per second. However, when the impulse response length increases to 50, the update rate drops to 1,084 Hz, reducing the system's ability to handle higher-frequency signals effectively. This decrease in update rate limits the bandwidth of the system, as a lower processing speed means the algorithm cannot accurately track or respond to rapid signal changes, making it less suitable for high-frequency applications.

When comparing execution speed, using long data types instead of floats generally results in faster processing due to simpler integer arithmetic and reduced computational overhead. In our case, the FIR algorithm executed significantly faster with long data types, allowing for a higher update rate and improved real-time performance. The fastest bandwidth we achieved was 3,030 Hz when using floats with an impulse response length of 20, but using long data types could potentially increase this speed even further. However, while integer arithmetic is faster, floating-point calculations provide more precise results, which is crucial in DSP applications that require accuracy. Even though using long data types may introduce errors due to truncation and reduced precision, they may still be useful in applications where speed is more critical than absolute accuracy, such as real-time signal monitoring.

III. CONCLUSION

This study demonstrates how each data type in DSP systems directly would govern the tradeoffs between accuracy, execution speed, and memory efficiency.

These results emphasize the importance of tailoring the data types one makes use of to the application specific requirements in order to preserve and simultaneously ensure a balance of computational efficiency and integrity of data in what is displayed to a reader.